

Vehicle Speed Estimation via Multi-Object Tracking and Planar Homography

1 Pipeline Overview

The pipeline consists of five sequential stages applied to every frame of a video captured at known frame rate f_{fps} :

1. **Tracking:** assigning a unique, persistent identity k and bounding box to each vehicle via BoT-SORT.
2. **Reference extraction:** extracting the ground-contact point of each tracked vehicle.
3. **Perspective projection:** mapping the 2D pixel coordinate to a 2D real-world metric coordinate via homography.
4. **Plausibility gating:** rejecting displacements that imply a physically impossible speed, treating them as tracker discontinuities rather than real motion.
5. **Speed computation & smoothing:** calculating displacement over a sliding window and applying an Exponential Moving Average (EMA) to suppress detection jitter.

2 Algorithmic Formulation

2.1 Reference Point Localization

For a bounding box defined by $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$, the chosen reference point is the **ground-contact centroid** (bottom-center of the box):

$$p_k^{(t)} = \left(\frac{x_{\min} + x_{\max}}{2}, y_{\max} \right) \quad (1)$$

Justification: the homography of Section 2.2 is valid only for points lying on the calibration plane (the road surface). The bottom-center of the box is the closest 2D approximation to the vehicle’s true wheel-to-road contact location available from an axis-aligned bounding box.

2.2 Perspective Calibration (Homography)

A homography matrix $H \in \mathbb{R}^{3 \times 3}$ maps a homogeneous pixel coordinate $(u, v, 1)^T$ to a world coordinate in meters:

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = H \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}, \quad (X, Y) = \left(\frac{x'}{w}, \frac{y'}{w} \right) \quad (2)$$

Why four points suffice. Fixing $h_{33} = 1$ leaves 8 unknown entries in H . Each known correspondence $(u_i, v_i) \mapsto (X_i, Y_i)$ contributes two linear equations:

$$X_i(h_{31}u_i + h_{32}v_i + 1) = h_{11}u_i + h_{12}v_i + h_{13} \quad (3)$$

$$Y_i(h_{31}u_i + h_{32}v_i + 1) = h_{21}u_i + h_{22}v_i + h_{23} \quad (4)$$

Four correspondences yield exactly 8 equations in 8 unknowns, solved directly by the OpenCV function `getPerspectiveTransform`. The four reference points and their real-world correspondences (a known lane width and segment length) are fixed once per camera setup, as shown in Listing 1.

2.3 Plausibility Gating

A tracker identity switch or a re-identification failure following occlusion produces a discontinuous jump in $P_k^{(t)}$ that is not real vehicle motion. Such a jump is detected *before* it enters the trajectory history, using the displacement to the immediately preceding point:

$$\text{reject if } \frac{d_k(t-1, t)}{1/f_{\text{fps}}} \times 3.6 > v_{\text{max}} \quad (5)$$

On rejection, the implementation clears the identity’s history rather than discarding only the offending point, since a corrupted track should not silently resume averaging once the discontinuity has occurred.

2.4 Speed Calculation and Temporal Smoothing

Once at least $w = 15$ points of (gated) history exist for identity k , the Euclidean ground-plane displacement across the window is:

$$d_k(t-w, t) = \sqrt{\left(X_k^{(t)} - X_k^{(t-w)}\right)^2 + \left(Y_k^{(t)} - Y_k^{(t-w)}\right)^2} \quad (6)$$

with elapsed time taken from the video’s native frame rate rather than wall-clock processing time:

$$\Delta t = \frac{t - (t-w)}{f_{\text{fps}}} \quad (7)$$

giving the raw windowed speed in km/h:

$$v_k^{(t)} = \frac{d_k(t-w, t)}{\Delta t} \times 3.6 \quad (8)$$

which is smoothed across time with an Exponential Moving Average of factor $\alpha \in (0, 1]$:

$$\bar{v}_k^{(t)} = \alpha v_k^{(t)} + (1 - \alpha) \bar{v}_k^{(t-1)}, \quad \bar{v}_k^{(t_0)} = v_k^{(t_0)} \text{ at first valid estimate} \quad (9)$$

Why smoothing is necessary even with a sliding window. The window in Equation (6) re-evaluates its oldest endpoint every frame, so the raw estimate $v_k^{(t)}$ still changes from frame to frame even when the vehicle moves at constant speed, due to bounding-box jitter at both window endpoints. The EMA of Equation (9) is therefore applied on top of windowing, not in place of it.

3 Code Implementation

The implementation below follows the Ultralytics YOLO + BoT-SORT structure used for detection and tracking, with the homography calibration, plausibility gate, and EMA smoothing integrated directly into the per-frame loop.

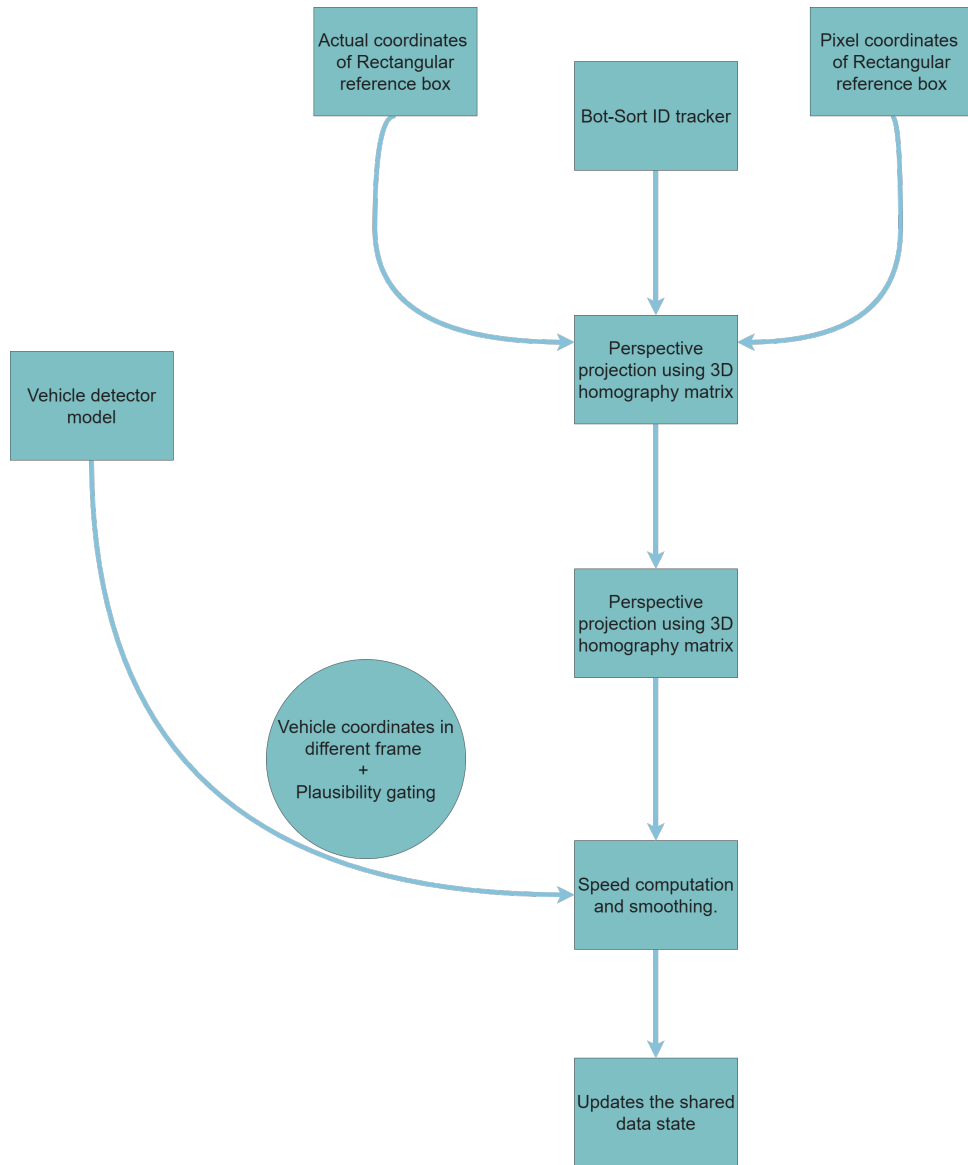


Figure 1: Workflow of the vehicle speed estimation pipeline.

3.1 Tracker Configuration and Homography Calibration

```

1 import cv2
2 import yaml
3 import numpy as np
4 from collections import defaultdict, deque
5 from ultralytics import YOLO
6 from ultralytics.data.loaders import get_best_youtube_url
7
8 # 1. Custom Tracker YAML
9 custom_tracker_path = "custom_botsort.yaml"
10 tracker_config = {
11     "tracker_type": "botsort",
12     "track_high_thresh": 0.25,
13     "track_low_thresh": 0.1,
14     "new_track_thresh": 0.25,
15     "track_buffer": 60,
16     "match_thresh": 0.8,

```

```

17     "fuse_score": True,
18     "gmc_method": "sparseOptFlow",
19     "proximity_thresh": 0.5,
20     "appearance_thresh": 0.8,
21     "with_reid": True
22 }
23
24 with open(custom_tracker_path, "w") as f:
25     yaml.dump(tracker_config, f)
26
27 # 2. Load the Model
28 model = YOLO("yolov8n.pt") #can change the model
29
30 # 3. Get Video Stream URL
31 video_url = get_best_youtube_url(
32     "https:your_url",
33     method="yt-dlp"
34 )
35
36 # 4. Video Setup
37 cap = cv2.VideoCapture(video_url)
38 if not cap.isOpened():
39     print("Error: Could not open video stream.")
40     exit()
41
42 w = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
43 h = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
44 fps = int(cap.get(cv2.CAP_PROP_FPS))
45 if fps == 0:
46     fps = 30
47
48 out_path = "tracked_speed_output.mp4"
49 fourcc = cv2.VideoWriter_fourcc(*"mp4v")
50 writer = cv2.VideoWriter(out_path, fourcc, fps, (w, h))
51
52 # --- Homography calibration: pixel polygon -> real-world rectangle ---
53 # IMPORTANT: replace these with coordinates matching a real rectangle
54 # on the road in your specific video.
55 src_points = np.array([
56     [100, 600], # Top-left pixel
57     [400, 600], # Top-right pixel
58     [500, 900], # Bottom-right pixel
59     [50, 900] # Bottom-left pixel
60 ], dtype=np.float32)
61
62 real_world_width = 3.5 # meters
63 real_world_length = 10.0 # meters
64
65 dst_points = np.array([
66     [0, 0],
67     [real_world_width, 0],
68     [real_world_width, real_world_length],
69     [0, real_world_length]
70 ], dtype=np.float32)
71
72 transform_matrix = cv2.getPerspectiveTransform(src_points, dst_points)
73
74 MAX_PLAUSIBLE_KMH = 200.0 # Eq. (4) gating threshold

```

```

75 EMA_ALPHA = 0.3 # Eq. (6) smoothing factor
76
77 track_history = defaultdict(lambda: deque(maxlen=fps))
78 smoothed_speed = {}

```

Listing 1: Tracker configuration and homography setup

3.2 Per-Frame Tracking and Speed Computation

```

1 frame_count = 0
2
3 try:
4     while cap.isOpened():
5         success, frame = cap.read()
6         if not success:
7             break
8
9         frame_count += 1
10
11         # tracker=custom_tracker_path wires in the BoT-SORT config built above.
12         # Without this argument, model.track() silently falls back to the
13         # default tracker and the tuned thresholds are never applied.
14         results = model.track(
15             frame, persist=True, verbose=False, tracker=custom_tracker_path
16         )
17         annotated_frame = frame.copy()
18
19         if results[0].boxes is not None and results[0].boxes.id is not None:
20             boxes = results[0].boxes.xyxy.cpu().numpy()
21             track_ids = results[0].boxes.id.cpu().numpy().astype(int)
22
23             for box, track_id in zip(boxes, track_ids):
24                 x1, y1, x2, y2 = box
25
26                 # Eq. (1): ground-contact reference point
27                 cx = (x1 + x2) / 2
28                 cy = y2
29
30                 # Eq. (2): pixel -> real-world via homography
31                 point_2d = np.array([[cx, cy]], dtype=np.float32)
32                 point_3d = cv2.perspectiveTransform(point_2d, transform_matrix)
33                 real_x, real_y = point_3d[0][0]
34
35                 # Eq. (4): plausibility gate, applied before appending history
36                 history = track_history[track_id]
37                 if len(history) > 0:
38                     last_x, last_y, last_frame = history[-1]
39                     step_distance = np.sqrt((real_x - last_x) ** 2 + (real_y - last_y)
40                                             ** 2)
41                     step_frames = frame_count - last_frame
42                     if step_frames > 0:
43                         implied_kmh = (step_distance / (step_frames / fps)) * 3.6
44                         if implied_kmh > MAX_PLAUSIBLE_KMH:
45                             history.clear()
46                             smoothed_speed.pop(track_id, None)
47
48                 history.append((real_x, real_y, frame_count))

```

```

48     speed_kmh = smoothed_speed.get(track_id, 0.0)
49     if len(history) >= 15:
50         old_x, old_y, old_frame = history[0]
51
52         # Eq. (5): windowed Euclidean displacement
53         distance_meters = np.sqrt((real_x - old_x) ** 2 + (real_y - old_y)
54                                   ** 2)
55
56         # Eq. (6): elapsed time from native fps
57         time_seconds = (frame_count - old_frame) / fps
58
59         if time_seconds > 0:
60             speed_ms = distance_meters / time_seconds
61             raw_speed_kmh = speed_ms * 3.6
62
63             # Eq. (7): EMA smoothing
64             prev = smoothed_speed.get(track_id)
65             speed_kmh = (
66                 raw_speed_kmh if prev is None
67                 else EMA_ALPHA * raw_speed_kmh + (1 - EMA_ALPHA) * prev
68             )
69             smoothed_speed[track_id] = speed_kmh
70
71     cv2.rectangle(annotated_frame, (int(x1), int(y1)), (int(x2), int(y2)),
72                  (0, 255, 0), 2)
73     cv2.putText(annotated_frame, f"ID:_{track_id}|_{speed_kmh:.1f}_km/h",
74                (int(x1), int(y1) - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,
75                255, 255), 2)
76
77     writer.write(annotated_frame)
78     cv2.imshow("Speed_Tracking", annotated_frame)
79
80     if cv2.waitKey(1) & 0xFF == ord("q"):
81         break
82
83 finally:
84     cap.release()
85     if writer is not None:
86         writer.release()
87     cv2.destroyAllWindows()
88     print(f"Saved_speed_tracking_video_to_{out_path}")

```

Listing 2: Main tracking loop with plausibility gating and EMA smoothing

4 Sources of Error and Limitations

- **Planarity assumption:** vehicles with high ground clearance, or camera angles that map onto a higher point on the vehicle body, introduce reprojection error, since the reference point of Equation (1) diverges from the true road-plane contact point as vehicle height increases relative to camera elevation.
- **Frame rate variability:** variable frame rate (VFR) source video invalidates the constant- f_{fps} assumption in Equation (7). With VFR sources, per-frame timestamps from the video container should replace frame-index-based timing.

- **Residual ID-switch risk:** the plausibility gate of Section 2.3 catches large, obviously implausible jumps, but a moderate ID switch between two adjacent, similarly-positioned vehicles can fall below $v_{\max}$ and still corrupt a window silently. This is a known residual risk rather than a fully solved case.
- **Calibration sensitivity:** error in the manually measured real-world reference distances used in Section 2.2 propagates directly into every downstream speed estimate; recalibration is required if the camera is repositioned.

5 References

- https://github.com/roboflow/supervision/tree/develop/examples/speed_estimation
- <https://github.com/roboflow/notebooks/blob/main/notebooks/how-to-estimate-vehicle-speed.ipynb>
- <https://blog.roboflow.com/estimate-speed-computer-vision/>